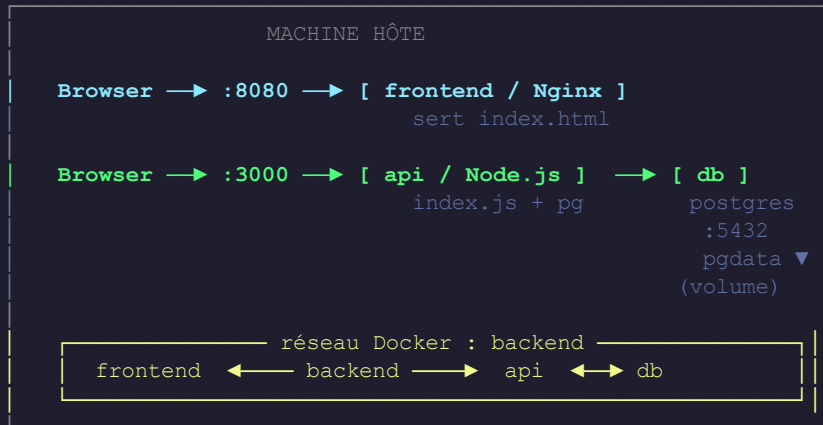


TP2 — Docker Compose

Frontend Nginx · API Node.js + pg · PostgreSQL

Ducarois Mathis

Architecture de la stack



Service	Image	Port hôte	Rôle
frontend	Dockerfile (nginx:alpine)	:8080	Sert index.html au navigateur via Nginx
api	Dockerfile.api (node:18-alpine)	:3000	API REST Node.js — lit/écrit dans PostgreSQL
db	postgres:15-alpine	interne	Base de données PostgreSQL — données persistées dans pgdata

Le Dockerfile fourni est celui du **frontend (Nginx)**. Il faut créer un **Dockerfile.api** séparé pour l'API Node.js qui utilise la dépendance **pg** (client PostgreSQL).

```
FROM node:18-alpine

WORKDIR /app

# Optimisation du cache : dépendances d'abord
COPY package*.json ./
RUN npm install --omit=dev

# Code source copié APRÈS les dépendances
COPY index.js ./

# Utilisateur non-root fourni par l'image node
USER node

EXPOSE 3000
CMD ["node", "index.js"]
```

Explications — Dockerfile.api

Instruction	Pourquoi
<code>FROM node:18-alpine</code>	Alpine = image de base ~50 MB au lieu de ~1 GB pour node:18. La dépendance pg compile nativement mais le binaire précompilé (pg-native) n'est pas utilisé ici — pg pure JS fonctionne sur Alpine.
<code>COPY package*.json → npm install</code>	On copie package.json AVANT index.js pour exploiter le cache Docker : si index.js change mais que les dépendances restent identiques, npm install sera CACHED et le rebuild sera instantané.
<code>RUN npm install --omit=dev</code>	Installe uniquement les dépendances de production (pg). nodemon et autres outils de dev sont exclus → image finale plus légère et surface d'attaque réduite.
<code>USER node</code>	L'image node:18-alpine fournit un utilisateur node (uid=1000). En basculant sur cet utilisateur, le processus ne tourne pas en root. Exigé par les bonnes pratiques et certains clusters Kubernetes.
<code>CMD ["node", "index.js"]</code>	Lancement direct sans shell intermédiaire (forme exec, pas shell). Ainsi le processus node reçoit directement les signaux SIGTERM/SIGINT → arrêt propre du container avec docker compose down.

services:

```
# — 1. PostgreSQL —————
db:
  image: postgres:15-alpine
  restart: unless-stopped
  environment:
    POSTGRES_DB:      ${DB_NAME}
    POSTGRES_USER:    ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - pgdata:/var/lib/postgresql/data
  healthcheck:
                                # attend que pg soit prêt
    test: ["CMD-SHELL","pg_isready -U ${DB_USER} -d ${DB_NAME}"]
    interval: 5s
    timeout: 5s
    retries: 10
    start_period: 15s
  networks:
    - backend

# — 2. API Node.js —————
api:
  build:
    context: .
    dockerfile: Dockerfile.api
  restart: unless-stopped
  environment:
    PORT:      ${API_PORT:-3000}
    DB_HOST:    db          # résolution DNS Docker
    DB_PORT:    5432
    DB_NAME:    ${DB_NAME}
    DB_USER:    ${DB_USER}
    DB_PASSWORD: ${DB_PASSWORD}
  ports:
    - "${API_PORT:-3000}:3000"
  depends_on:
    db:
      condition: service_healthy
  networks:
    - backend

# — 3. Frontend Nginx —————
frontend:
  build:
    context: .
    dockerfile: Dockerfile
  restart: unless-stopped
  ports:
    - "${FRONTEND_PORT:-8080}:80"
  depends_on:
    - api
  networks:
    - backend

networks:
  backend:
                                # réseau partagé entre les 3
    driver: bridge
```

```
volumes:
  pgdata:                                # données Postgres persistées
```

Concepts clés expliqués

Concept	Explication
DB_HOST: db(résolution DNS)	L'API se connecte à la BDD via le nom du service : db. Dans un réseau Docker Compose, chaque service est joignable par son nom (ex: http://api:3000, psql://db:5432). Docker maintient un DNS interne qui résout ces noms en IP automatiquement. → Pas besoin de connaître l'IP du container !
depends_on: condition:service_healthy	Sans healthcheck, depends_on démarre l'api dès que le container db est démarré, mais PostgreSQL peut mettre 5-10 secondes à être vraiment prêt à accepter des connexions. Avec condition: service_healthy, Docker attend que le healthcheck pg_isready réussisse avant de démarrer l'API. Évite l'erreur "connection refused" au démarrage.
volumes: pgdata	Sans volume, toutes les données PostgreSQL sont perdues à chaque docker compose down. Le volume nommé pgdata persiste même après arrêt des containers. docker compose down → containers supprimés, données conservées docker compose down -v → containers + volume supprimés (reset complet)
networks: backend	Les 3 services sont sur le même réseau bridge backend. Ils peuvent se parler par leur nom de service (api → db) mais sont isolés du reste de la machine hôte. Seuls les ports déclarés dans ports: sont accessibles depuis l'hôte. → db n'expose pas de port : impossible d'y accéder depuis l'extérieur.
restart: unless-stopped	En cas de crash de l'app ou de redémarrage de la machine, Docker relance automatiquement les containers. La valeur unless-stopped signifie : relancer toujours SAUF si l'admin a explicitement arrêté le container avec docker compose stop.

Fichier .env — Variables d'environnement

```
# — Base de données —————
DB_NAME=tp2db
DB_USER=tp2user
DB_PASSWORD=tp2password

# — Ports hôte —————
API_PORT=3000
FRONTEND_PORT=8080
```

Pourquoi utiliser un fichier .env ?

- Séparation config/code : le code ne contient aucune valeur sensible en dur.
- Flexibilité : un seul docker-compose.yml fonctionne en dev, recette et prod (seul .env change).
- Sécurité : le .env est exclu de Git → les secrets ne fuient pas dans l'historique.
- Surcharge facile : docker compose --env-file .env.prod up pour pointer sur la prod.

Fichier .dockerignore

```
node_modules/
.env
.env.*
.git/
Dockerfile*
docker-compose*.yml
*.md
coverage/
```

Règle	Raison d'exclusion
node_modules/	Le node_modules local peut être compilé pour un OS différent (macOS vs Alpine Linux). npm install dans le Dockerfile recrée les bons binaires pour l'image.
.env / .env.*	Contient les mots de passe de la BDD. Embarqué dans l'image → lisible par docker inspect ou docker history par n'importe quel développeur du projet.
.git/	Historique Git complet = inutile + potentiellement des commits avec d'anciens secrets. Peut alourdir le contexte de build de plusieurs centaines de MB.
Dockerfile* / docker-compose*	Ces fichiers de configuration ne doivent pas se retrouver dans l'image finale (ils appartiennent à l'infra, pas à l'app).

Démarrage et tests

```
# 1. Démarrer toute la stack (build + up)
docker compose up --build

# 2. Vérifier que les 3 containers tournent
docker compose ps
#      NAME                STATUS
#      tp2-db-1             healthy
#      tp2-api-1            running
#      tp2-frontend-1      running

# 3. Tester l'API directement
curl http://localhost:3000/health
#    → {"status":"ok"}

curl http://localhost:3000/messages
#    → [] (vide au départ)

curl -X POST http://localhost:3000/messages \
  -H "Content-Type: application/json" \
  -d '{"texte": "Hello Docker Compose!"}'
#    → {id:1, texte: '...', created_at: '...'}

# 4. Ouvrir le frontend
open http://localhost:8080
#    → page HTML avec formulaire + messages
```

Commandes de diagnostic

Commande	Ce qu'elle vérifie
<code>docker compose logs -f api</code>	Logs en temps réel de l'API (connexion BDD, erreurs)
<code>docker compose logs db</code>	Logs PostgreSQL (utile pour debug de connexion)
<code>docker compose exec db psql -U tp2user -d tp2db</code>	Ouvrir un shell SQL dans la BDD
<code>docker volume ls</code>	Lister les volumes → vérifier que pgdata existe
<code>docker compose down -v</code>	Arrêter + supprimer volumes (reset complet)
<code>docker compose restart api</code>	Redémarrer uniquement l'API sans tout couper
<code>docker network inspect tp2_backend</code>	Voir les IPs des containers sur le réseau